

Proyecto:

Gambling Casino

Samuel Londoño Mayorga

Jhosef Alejandro Rojas Durán

Juan Alejandro Castillo López

Paula Andrea Gutierrez Bonilla

Facultad de ingeniera, Universidad Libre de Colombia

Ingeniería de sistemas, Programación I

Andrés Felipe Castano Rubiano

9/28/2026

Introducción:	2
Planteamiento del Problema Solucionado	2
Objetivos:	3
Estructura de la aplicación	3
Jerarquía y diagrama de clases:	4
Justificación de decisiones de diseño:	5
Explicación de la Aplicación de Conceptos de Programación Orientada a Objetos (POO)	6
Funcionamiento de los juegos.....	8
Requisitos:	9
LINK A GITHUB.....	9

Introducción:

Este proyecto consiste en el desarrollo de una aplicación en Java que simula el funcionamiento de un casino, mostrando juegos de azar: Tragaperras, Ruleta y Dados. Este sistema permite crear varios usuarios y administrar sus saldos a lo largo de cada sesión de juego mientras apuestan.

Se utilizó la Programación Orientada a Objetos (POO) utilizando clases abstractas, herencia, polimorfismo e interfaces.

Planteamiento del Problema Solucionado

El proyecto soluciona la necesidad de simular un juego de casino que cuenta con tres juegos diferentes: Dados, Tragaperras y Ruleta,

El problema que resuelve este sistema es el control de las apuestas y el saldo de cada jugador/usuario. Donde se busca evitar que el juego continúe cuando el saldo de un jugador llega a ceros o valores negativos. Además, se necesitaba integrar varios juegos en una sola aplicación para que el usuario no tenga que abrir más programas para cada juego.

Objetivos:

Desarrollar una aplicación interactiva en Java que simule un juego de casino con los minijuegos Tragaperras, Dados y Ruleta, aplicando los conceptos de Programación Orientada a Objetos (POO) y usando una interfaz para gestionar los jugadores.

Objetivos Específicos:

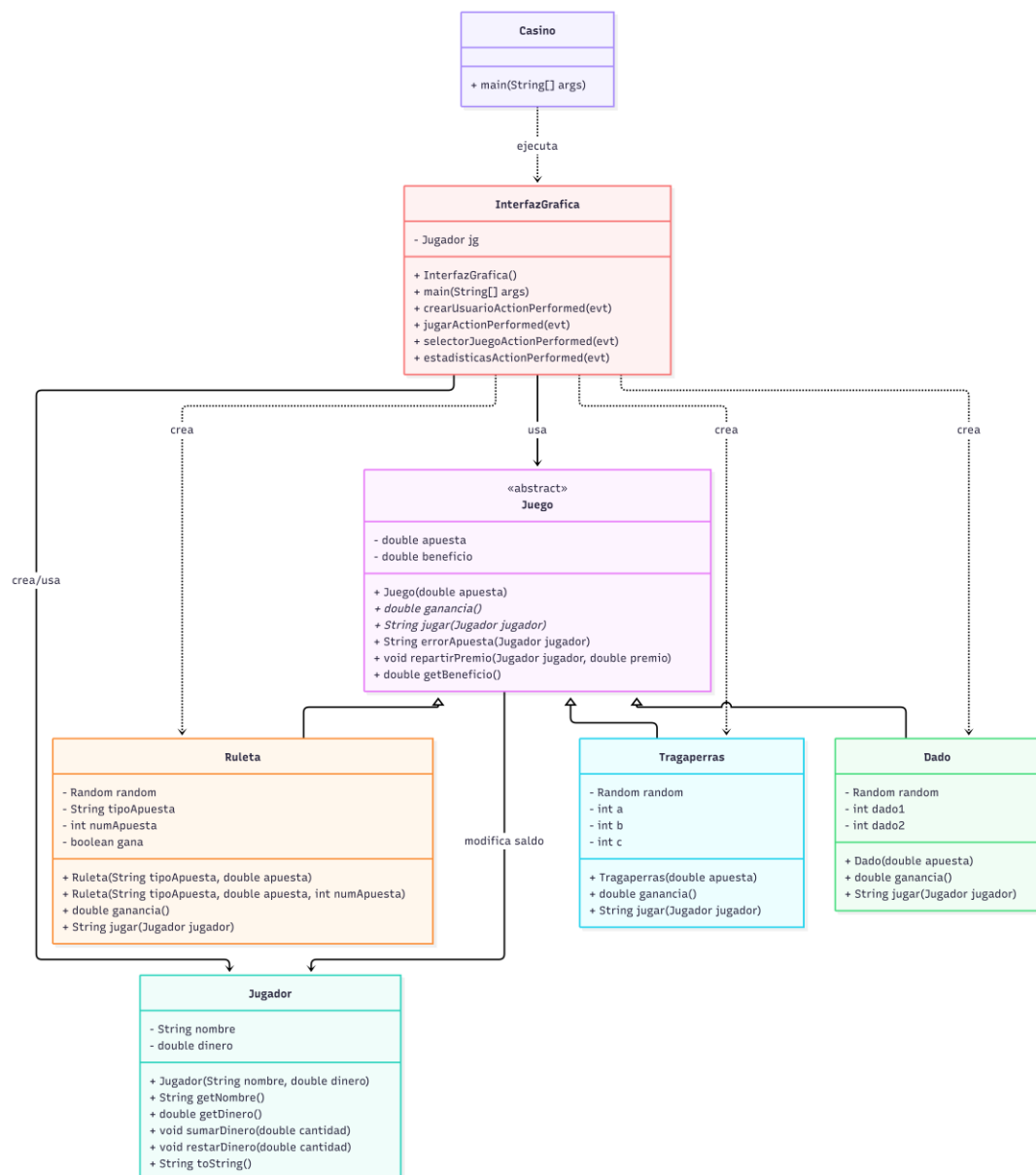
- Diseñar una arquitectura de clases utilizando abstracción, encapsulamiento, herencia, polimorfismo e interfaces para representar a los jugadores y la lógica interna de los juegos.
- Crear un mecanismo de control de errores mediante excepciones que impida realizar apuestas inválidas sin que el sistema deje de funcionar.
- Separar la interfaz gráfica de la lógica y código en donde únicamente se capture las acciones del usuario y muestre las respuestas generadas por los objetos del juego.

Estructura de la aplicación

Componente	Tipo	Responsabilidad
InterfazGrafica	Jframe	Gestiona: controles, java Swing, crea jugadores, permite seleccionar los

		juegos y muestra los resultados
Jugador	Clase	Crea el objeto jugador que tiene encapsulados si nombre y dinero, además, permite restar y sumar su saldo
Juego	Clase abstracta	Contiene la apuesta, el beneficio, la validación de la apuesta, el reparto del premio, además, crea métodos abstractos para jugar y obtener la ganancia
Ruleta	Subclase de juego	Genera un numero aleatorio de 1 a 36 y permite apostar por el tipo de valor que saldrá: par, impar o número específico
Tragaperras	Subclase de juego	Genera tres números aleatorios de 0 a 9 que deben coincidir o ser una secuencia numérica para ganar
Dado	Subclase de juego	Genera dos números aleatorios de 0 a 6 cuya suma debe dar 7 para ganar
ErrorApuesta	Interfaz	Define el contrato necesario para validar la apuesta de un jugador

Jerarquía y diagrama de clases:



Justificación de decisiones de diseño:

El proyecto consiste en un casino desarrollado en java, donde se busca separar la lógica de los juegos de la interfaz gráfica. El usuario puede crear jugadores, seleccionar jugado, y elegir un juego, el jugador tiene que realizar una apuesta después de esto el jugador podrá verificar el resultado en la pantalla y se mostrará el saldo actualizado.

Para la interfaz utilizamos java swing, porque nos permite construir una aplicación grafica con botones, campos de texto, listas desplegables y el área de resultados, creando un archivo JFrame. Con esta interfaz, se nos permite centralizar los diferentes juegos en una sola ventana lo que facilita la interacción del usuario con el sistema.

La interfaz también se encarga de recibir y validar los datos que introduzca el usuario, como lo son el jugador seleccionado y el valor de la apuesta. Dependiendo del juego escogido, la interfaz puede mostrar diferentes opciones o controles correspondientes al tipo de apuesta.

Para el programa, se crearon diferentes juegos que pertenecen a casinos, esto, quiere decir que existe una variedad de tres juegos donde se evidencia el uso de la herencia y el polimorfismo pues cada juego tiene sus propias reglas.

Explicación de la Aplicación de Conceptos de Programación Orientada a Objetos (POO)

- **Abstracción:** El código implementa la clase abstracta "Juego" el cual modela las características generales de cada juego de azar como la apuesta y premios.

```
public class Dado extends Juego{  
    private Random random = new Random();  
    private int dado1, dado2;  
  
    ...}
```

```
public abstract class Juego implements ErrorApuesta{  
  
    protected double beneficio = 0;  
    protected double apuesta = 0;  
  
    ....}
```

- **Encapsulamiento:** Se aplica en todas las clases especialmente en Jugador Ruleta Dados y Tragaperras mediante atributos privados como private String nombre. También el acceso y control a el saldo es posible gracias a los métodos de sumarDinero(), restarDinero() y getDinero().

```
public class Jugador {  
    private String nombre;  
    private double dinero;  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public double getDinero() {  
        return dinero;  
    }  
}
```

- **Herencia:** Se evidencia el uso de esto en las clases de Dado, Ruleta y Tragaperras cuando extienden de Juego la gestión de las apuestas.

```
public class Ruleta extends Juego { ... }
public class Tragaperras extends Juego { ... }
public class Dado extends Juego { ... }
```

- **Polimorfismo:** Se usa el “@Override” para que cada clase redefina sus métodos abstractos heredados “jugar(Jugador jugador) y ganancia() según cada regla de juego.

```
@Override
public double ganancia() {
    if (dado1 + dado2 == 7) {
        return apuesta * 5;
    }
    return 0;
}
```

- **Interfaz:** ErrorApuesta define el método errorApuesta(Jugador jugador), la cual es implementada por la clase juego, que, a su vez la hereda a todas sus demás clases juegos al ser una validación común que todas las demás clases necesitan realizar.

```
public interface ErrorApuesta {
    void errorApuesta(Jugador jugador)
    throws IllegalArgumentException;
}
```

- **Manejo de excepciones:** Todos los juegos implementan try..catch para que cada juego pueda capturar la excepción de uno de los tres casos contemplados:
 - Jugador inexistente.
 - Apuesta menor o igual a 0.
 - Apuesta superior al saldo disponible.

```
try {
    errorApuesta(jugador);
} catch (IllegalArgumentException e) {
    return e.getMessage();
}
```

- **Interfaz Gráfica:** Funciona como controlador del sistema en donde el usuario interactúa con una interfaz que contiene botones y selectores para crear el Jugador y seleccionar el juego, como se mencionó anteriormente, fue construida con java Swing en un formulario JFrame:

Crear usuario

Nombre

Saldo

Usuario:

Juego:

Apuesta:

Tipo de Ruleta

- El usuario puede ingresar un nombre y saldo, al presionar "Crear" se crea un jugador, el cual es almacenado en un ArrayList.
- Se puede seleccionar un jugador, previamente creado y guardado en el ArrayList, posteriormente, se selecciona uno de los tres juegos (Ruleta, Tragaperras o Dado), se selecciona el monto a apostar y se le da al botón "Jugar".
- Finalmente, en el Text Area se muestra el resultado de la apuesta, el cual puede ser ganar, perder, o en última instancia un error generado por no ingresar algún campo correctamente.

Funcionamiento de los juegos

Juego	Reglas	Ganancia
-------	--------	----------

Ruleta	Genera un numero aleatorio de 1 a 36 y permite apostar por el tipo de valor que saldrá: par, impar o número específico	Par o impar: premio=apuesta+50% de la apuesta. Numero específico: Premio= apuesta + 10 veces la apuesta
Tragaperras	Genera tres números aleatorios de 0 a 9 que deben coincidir o ser una secuencia numérica para ganar	Tres valores iguales: Premio = 10 veces la apuesta Secuencia: Premio = 5 veces la apuesta
Dado	Genera dos números aleatorios de 0 a 6 cuya suma debe dar 7 para ganar	Premio = 5 veces la apuesta

Requisitos:

- JDK de Java con soporte para Swing (Java 8 o superior).
- Se recomienda usar NetBeans.

LINK A GITHUB

[ZedEnjoyer/Java-NetBeans-Programacion-1](https://github.com/ZedEnjoyer/Java-NetBeans-Programacion-1)